

Git Slay

CDS project

S. Lott¹, A. Haizmann¹

December, 2024

Problem statement

When this project started, our initial idea was to replicate part of GitHub's functionalities. However, we quickly realized that recreating an already existing system would not only be redundant but would also fail to meet the innovative and exploratory goals of this project. Instead, we shifted our focus to addressing a key limitation in GitHub's design: its reliance on a centralized architecture.

While GitHub is a highly robust platform that has solved many challenges related to file storage, version control, and access rights, its centralized nature introduces potential constraints. For instance, all branches and repositories ultimately depend on a single point of access: GitHub's servers. This architecture, while efficient and widely adopted, can create bottlenecks in scalability, reduce fault tolerance, and limit user autonomy. Additionally, the reliance on a centralized system may pose risks during server outages or in situations where decentralization could provide greater flexibility.

This led us to explore an alternative approach: reimagining a version control system as a fully decentralized architecture. By doing so, we aim to investigate how multiple access points could enhance resilience, autonomy, and collaboration in distributed systems. This shift aligns with the project's goal of exploring innovative system designs and addressing real-world limitations, whilst using distributed and or concurrent systems.

Contents

1	State of the art	2
2	Presentation of the solution	2
3	Validation/evaluation of the solution	3
4	Discussion of the results	4

1. State of the art

The landscape of version control and collaboration systems is vastly dominated by centralized platforms, with GitHub standing as the most popular example. GitHub’s architecture relies on centralized servers that store repositories, coordinate branches and conflicts, and handle user authentication. However, this centralized approach introduces potential limitations, such as dependency on the availability and reliability of GitHub’s servers and the lack of autonomy for users over their repositories.

Alternative solutions, such as GitLab and Bitbucket, adopt similar centralized models, with slight variations in features and deployment options (e.g., self-hosting). However, these platforms do not fundamentally depart from GitHub’s reliance on a single point of access.

On the other hand, fully decentralized systems, such as IPFS (InterPlanetary File System) and Git-based peer-to-peer solutions, represent a different paradigm. These systems eliminate the reliance on centralized servers by allowing files or repositories to be stored and accessed across a network of peers. And while these approaches improve resilience and autonomy, they introduce challenges such as network optimization, consistency management, and access control in distributed environments.

Research on distributed systems provides a rich foundation for exploring the multiple access points aspect of this project. For example:

- **Optimizing Network Use:** Protocols such as gossip protocols or peer-to-peer overlays can efficiently propagate data in a decentralized network.
- **Message Handling:** Techniques for achieving consistency and ensuring message delivery in distributed systems are essential when designing systems with multiple access points.
- **Conflict Resolution:** Distributed version control systems, like Git, have explored approaches to handle conflicting changes across multiple copies of a repository.

Whilst a comprehensive review of these topics is beyond the scope of this report, they provide valuable context for the design and implementation of this project. By combining the strengths of GitHub’s version control capabilities with concepts from decentralized architectures, our project aims to explore an innovative solution that addresses the identified limitations of centralized systems.

2. Presentation of the solution

As previously stated, our approach is based on the possibility of having multiple users connected through a standard Internet network available to store and share files. This solution should allow users to join the network at any time and make their files available to anyone connected, as well as preventing mass inaccessibility if a single node fails. In addition, the idea is to use causal-order broadcasting to ensure ordering of communication.

In practice, we made a number of adaptations to scale down the project to fit a work frame that was manageable to complete in a reasonable amount of time. These adaptations are as follows: the files available through network are not updated for everyone when a new user joins the broadcasting group. This is because it is not an essential feature since users can simply refresh their page to have the updated available files. The broadcasting is done through a server that handles a user list and all calls between users. This was done because peer to peer communication requires more message handling, typically in the case of a client crashing, than if a central communicator waits or not for responses.

Now let’s go a bit more into details of how the application does function. When a client starts running, it connects to the broadcasting server via HTTP over IPv4 to be added to the clients list; it also checks for all files that are available locally to offer and verifies which are available in other users.

```
def home(conn, _params) do
  local_files = Client.Backend.list_local_files()
  all_files = Client.Backend.list_all_files()
  render(conn, :home, local_files: local_files,
    ↪ all_files: all_files)
end

def list_local_files() do
  for f <- Path.wildcard(@filesPath <> "user/*"),
    File.regular?(f),
    do: String.split(f, "/", parts: :infinity)
    ↪ |> List.last()
end

def list_remote_files() do
  for f <- Path.wildcard(@filesPath <>
    ↪ "commit/*"), File.regular?(f), into: %{} do
    f_name =
      String.split(List.last(String.split(f, "/",
        ↪ parts: :infinity)), @splitChars, parts:
        ↪ 2)
    |> hd()

    mr_ver = Enum.max(Map.get(@record, f_name))

    {f_name, mr_ver}
  end
end
```

The internal structure is as follows: the client and the server are two completely separate entities. The

server is its own application and, on the server side, the communication between the server and clients is handled through a router that calls the various functions on the API of the server so as to add a layer of abstraction and security.

```
post "/api/register" do
  {:ok, body, _conn} = Plug.Conn.read_body(conn)
  %{"ip" => ip} = Jason.decode!(body)
  client_id =
    → Server.CentralServer.register_client(ip)
  send_resp(conn, 200, Jason.encode!({:status:
    → "ok", client_id: client_id}))
end

def register_client(ip) do
  GenServer.call({:global, __MODULE__},
    → {:register_client, ip})
end
```

On the client side, the idea is fairly in-line with the server, as communications go through a router that calls page controllers that then split calls between the front and back-end. This allows the client application to have separation of tasks and isolation of code to apply encapsulation, with all the benefits that it entails.

The backend is coded in pure elixir and features a number of functionality acting as endpoints, contrary to most series done for this course. This decision was motivated by the reality that the application has network connection as well as internal communication and native process message passing would not as easily fit the various context of local and global network communication as endpoints do. The functionalities implemented are parsing and returning lists of available files, answering requests of available file versions, committing files and sending file data.

```
def commit(file) do
  remote_files = Application.get_env(:client,
    → :remote_files)
  mr_v = Map.get(remote_files, file, -1) + 1

  data =
    try do
      File.read!(@filesPath <> "user/#{file}")
    rescue
      File.Error -> ""
    end

  File.write(@filesPath <>
    → "commit/#{file}#{@splitChars}#{@mr_v}",
    → data)

  remote_files =
    if Map.has_key?(remote_files, file) do
      Map.put(remote_files, file, mr_v)
    else
      Map.put_new(remote_files, file, mr_v)
    end
end
```

```
Application.put_env(:client, :remote_files,
  → remote_files)
end
```

The front-end and routing is coded with the Phoenix framework, as there is no native network communication support nor web UI design in elixir. As was previously said, the architecture has ties with that of the server. It does, however, differ on some points, for example the additional layer of the controller between the router and the front or back-end.

```
scope "/api", ClientWeb do
  pipe_through :api

  post "/shutdown", PageController, :shutdown
  post "/list-local-files", PageController,
    → :list_local_files
  post "/file-list-response", PageController,
    → :file_list_response
end

def list_local_files(conn, _params) do
  file_list = Client.Backend.list_local_files()
  IO.puts("Client sending file list:
    → #{inspect(file_list)}")

  Client.Backend.
    send_file_list_to_server(file_list)
  json(conn, %{status: "ok", files: file_list})
end
```

The web page itself is generated by the Phoenix framework, which uses templates and classical html/css to render a visual.

3. Validation/evaluation of the solution

The evaluation of this project involves three key steps: verifying the conceptual strength of the solution, assessing how well the implementation fulfills the project's goals, and analyzing the alignment between the theoretical design and the actual codebase.

Step 1: Conceptual Strength of the Solution

The primary objective of this project was to create a distributed system with the following features:

- Multiple Access Points: Enable distributed access to data without relying on a single centralized server.
- File Storage: Provide a reliable mechanism for storing and retrieving files across the system.
- Asynchronous and Reliable Communication: Ensure communication between nodes is both asynchronous and reliable.
- No Single Point of Failure: Avoid central points of failure to improve resilience.

The conceptual solution meets these goals by leveraging a distributed architecture where multiple users act as potential access points for data. File storage is integral to the design, and causal order broadcasting ensures reliable message delivery without requiring time synchronization, fulfilling the asynchronicity requirement. Theoretical analysis confirms that the system avoids single points of failure at the data storage level, addressing the core problem effectively.

Step 2: Evaluation of Implementation

The next step is to assess how well the implemented solution aligns with these conceptual ideas.

- **Distributed Access Points:** The system allows users with the same file to act as access points, ensuring multiple data access paths. However, the presence of a central broadcasting server introduces a partial centralization point. While the server does not directly manage data, it remains a choke point for communication.
- **File Storage:** The implementation ensures files are pulled and stored by users, creating distributed replicas. This functionality aligns well with the project's goals.
- **Asynchronous and Reliable Communication:** Communication is asynchronous as no global synchronization or fixed timeline is required for message handling. The causal order broadcasting mechanism inherently guarantees reliability, meeting this requirement.
- **Avoiding Single Points of Failure:** While the distributed file storage eliminates single points of failure for data access, the broadcasting server remains a potential bottleneck. If it fails, communication halts, highlighting a limitation in the current design.

Step 3: Alignment Between Theory and Implementation

The final step involves examining how closely the implementation matches the theoretical design.

- **Distributed File Storage:** Fully realized in the coded solution. Files are stored locally by users and made accessible to others, satisfying the distributed access requirement.
- **Multiple Access Points:** Conceptually fulfilled but technically constrained by the central broadcasting server. The solution could be further enhanced by decentralizing the broadcasting mechanism to eliminate this choke point.

- **Asynchronous Communication:** Completely aligned with the theoretical design, as causal order broadcasting ensures reliable communication without timeline constraints.
- **No Single Point of Failure:** Partially achieved. While data access is resilient due to its distributed nature, communication is still dependent on the broadcasting server, introducing a potential failure point.

4. Discussion of the results

Considering the evaluation in the last section, the results are mitigated. A good portion of the features that were intended to be managed is covered, but there is still a good margin for improving the code base. However, to add a little nuance, the code basis that exists at the moment can be modified with a certain simplicity to accommodate most of the wanted features, the main driver of resource consumption being time in this case. This derives from the fact that most limitations can be removed by adding more messages and the corresponding handlers in the various routers and controllers; typically to add live updates of which user and file are available and having true multiple access points, adding correspondence handling in the client. Furthermore, the critique that can be made to the broadcasting server is not to be considered as a dichotomous situation, in the sense that if clients were responsible themselves for handling all messages as suggested just above, it would put a significant strain on them and they would need to implement more modules, for example a broadcasting module.

To add to this, creating the broadcasting group cannot be done without some knowledge of other users without losing significant time and resource in broadcasting on the whole world wide web. The choice made here to have a central server with a list of clients is therefore debatable but not necessarily a bad choice. To add a counter-point, an alternative solution could have been to make joining the group go through the server and then handle all communication through peer to peer message passing.

To summarize the debate, we would say that whilst the current version of the project has not fully met the ambition of our initial ideas, there is a good code basis for improving the code in the future with minimal modifications necessary.